Software Requirements and Design Document

For

Group <5>

Project

RaceTrack

Version 2.0

Authors:
Olivia Reiter
Julissa Su
Brooklyn Metzger
Yulissa Fu
Aleksandar “The Great” Stavreski
Alexander Hsieh

1. Overview (5 points)

Give a general overview of the system in 1-2 paragraphs (similar to the one in the project proposal).

RaceTrack is a web application that leverages machine learning to predict Formula 1 race outcomes using historical performance data, weather conditions, and race strategy parameters. The system provides users with race winner probability predictions for upcoming races, current driver and constructor championship standings, and a customizable race simulation tool that allows adjustment of variables such as circuit selection, weather conditions, and tire strategy. The application also includes a comprehensive historical data center covering race results from 2010 to present, and a newsroom aggregating current F1 updates from multiple sources.

The system is built on a React/TypeScript frontend, a Python FastAPI backend, and a PostgreSQL database, with machine learning components developed in Python using scikit-learn and XGBoost. It is designed as an open-access platform requiring no user authentication, prioritizing ease of use and data transparency over competitive or social features — distinguishing it from existing F1 prediction apps that focus primarily on fantasy leagues and point-based competitions. To keep predictions current across the active season, the system incorporates an automated data ingestion pipeline that pulls and stores 2026 race results as they become available, without requiring manual model retraining.

2. Functional Requirements (10 points)

*List the **functional requirements** in sentences identified by numbers and for each requirement state if it is of high, medium, or low priority. Each functional requirement is something that the system shall do. Include all the details required such that there can be no misinterpretations of the requirements when read. Be very specific about what the system needs to do (not how, just what). You may provide a brief design rationale for any requirement which you feel requires explanation for how and/or why the requirement was derived.*

- **FR-1: ML Prediction Service** *(High Priority)* The system shall predict race winner probabilities for a given race using a trained machine learning model, targeting 45–50% exact top-1 winner accuracy and 70% or greater top-3 podium prediction accuracy on held-out test data.
- **FR-2: Feature Calculation** *(High Priority)* The system shall calculate driver statistics — including win rate, recent form, and team performance average — from historical race data for use as model input features.
- **FR-3: Model Versioning** *(Medium Priority)* The system shall track and log the ML model version currently in use, accessible via backend configuration or API response metadata.
- **FR-4: Dashboard Display** *(High Priority)* The system shall display upcoming F1 race information on the Dashboard, including race name, circuit location, and a live countdown to the next scheduled race.
- **FR-5: Real-time UI Updates** *(Medium Priority)* The frontend Dashboard shall automatically refresh race data every five minutes in the background without requiring a full page reload.
- **FR-6: API Client Data Retrieval** *(High Priority)* The system shall retrieve F1 data — including next race, circuit details, and standings — through a centralized API client layer in the frontend.
- **FR-7: Historical Race Data Retrieval** *(High Priority)* The system shall fetch historical F1 race schedules, results, driver standings, and constructor standings from the Jolpica F1 API.

- **FR-8: Upcoming Race Schedule** *(High Priority)* The system shall expose an endpoint that returns the next scheduled race and the race immediately following it, used to populate the Dashboard race cards.
- **FR-9: Database Population** *(High Priority)* The system shall populate the database with race schedules and circuit information, preventing duplicate entries through validation checks on year and round number combinations.
- **FR-10: Idempotent Seeding** *(Medium Priority)* The system shall allow database seeding scripts to be run multiple times safely, updating existing records rather than creating duplicates.
- **FR-11: Race Simulation** *(High Priority)* The system shall provide a Simulator page that accepts user-defined parameters — including circuit, weather condition, tire strategy, and starting grid order — and returns a predicted race result by invoking the `simulate_race()` function of the trained ML model.
- **FR-12: Data Center Page** *(Medium Priority)* The system shall provide a Data Center page that allows users to browse historical race results, circuit information, and driver/constructor standings, consistent with the design specification.
- **FR-13: Newsroom Page** *(Medium Priority)* The system shall provide a Newsroom page that aggregates current F1 news from multiple external sources and displays articles in a filterable card-based layout.
- **FR-14: Automated Data Ingestion Pipeline** *(High Priority)* The system shall automatically pull and store 2026 race results from external APIs as each race completes, updating the database without requiring manual intervention or model retraining.

3. Non-functional Requirements (10 points)

*List the **non-functional requirements** of the system (any requirement referring to a property of the system, such as security, safety, software quality, performance, reliability, etc.) You may provide a brief rationale for any requirement which you feel requires explanation as to how and/or why the requirement was derived.*

- **NFR-1: Prediction Performance** *(High Priority)* The system shall generate race predictions and return a response to the frontend in under 2 seconds under normal operating conditions.
- **NFR-2: Model Reliability** *(High Priority)* The system shall handle ML model loading failures gracefully, returning a clear error message to the frontend rather than crashing the backend service.
- **NFR-3: Model Accuracy** *(High Priority)* The ML model shall achieve a minimum of 45% top-1 winner prediction accuracy and 70% top-3 podium prediction accuracy on held-out test data. These targets reflect the inherent unpredictability of F1 race outcomes and are intentionally conservative to ensure defensible, reproducible results.
- **NFR-4: Usability / Responsiveness** *(Low Priority)* The frontend interface shall be responsive, maintaining a readable and functional layout on both desktop and mobile screen sizes.
- **NFR-5: Frontend Reliability** *(High Priority)* The frontend API client shall gracefully handle network errors, retrying failed requests up to three times before displaying a user-facing error message.
- **NFR-6: API Rate Limiting** *(High Priority)* The system shall enforce a rate limit of no more than 2 calls per second to external APIs to prevent service abuse and comply with third-party usage policies.
- **NFR-7: Seeding Performance** *(Medium Priority)* The system shall complete database seeding of one full F1 season's worth of data in under 60 seconds on standard development hardware.
- **NFR-8: Cache Storage Efficiency** *(Low Priority)* The system shall store cached API responses in a structured file-based format that minimizes redundant external calls across application restarts.

- **NFR-9: Data Ingestion Reliability** *(Medium Priority)* The automated data ingestion pipeline shall handle API failures and partial responses without corrupting existing database records, logging errors for manual review when ingestion cannot complete successfully.

4. Use Case Diagram (10 points)

*This section presents the **use case diagram** and the **textual descriptions** of the use cases for the system under development. The use case diagram should contain all the use cases and relationships between them needed to describe the functionality to be developed. If you discover new use cases between two increments, update the diagram for your future increments.*
*Textual descriptions of use cases: **For the first increment, the textual descriptions for the use cases are not required.** However, the textual descriptions for all use cases discovered for your system are required for the second and third iterations.*

5. Class Diagram and/or Sequence Diagrams (15 points)

*This section presents a high-level overview of the anticipated system architecture using a **class diagram** and/or **sequence diagrams**.*

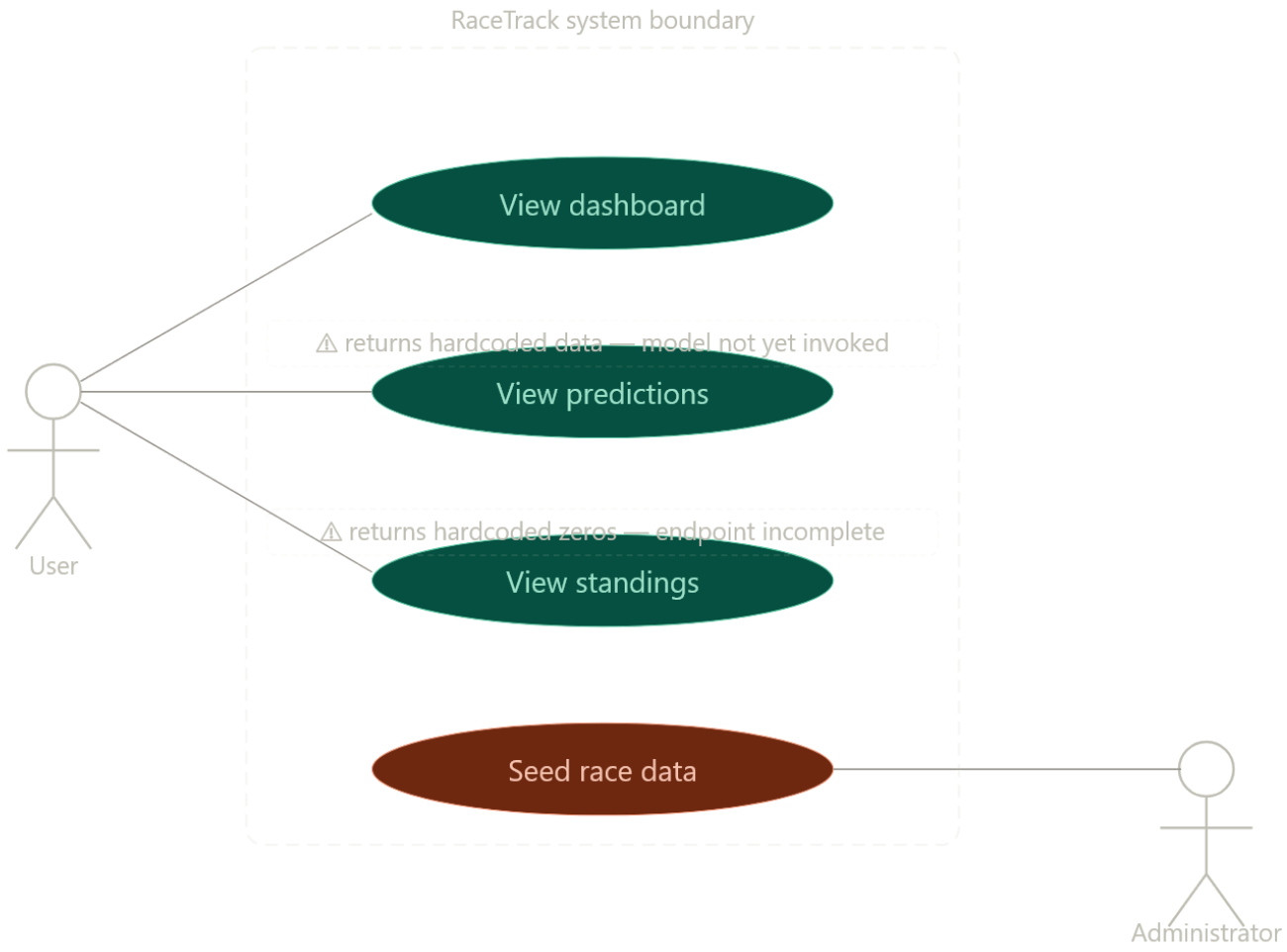
*If the main **paradigm** used in your project is **Object Oriented** (i.e., you have classes or something that acts similar to classes in your system), then draw the **Class Diagram of the entire system and Sequence Diagrams for the three (3) most important use cases in your system.***

*If the main **paradigm** in your system is **not Object Oriented** (i.e., you **do not** have classes or anything similar to classes in your system) then only draw **Sequence Diagrams**, **but for all the use cases of your system.** In this case, we will use a modified version of Sequence Diagrams, where instead of objects, the lifelines will represent the functions in the system involved in the action sequence.*

***Class Diagrams** show the **fundamental objects/classes** that must be modeled with the system to satisfy its requirements and **the relationships** between them. Each class rectangle on the diagram **must also include the attributes and the methods of the class** (they can be refined between increments). All the **relationships between classes and their multiplicity** must be shown on the class diagram.*

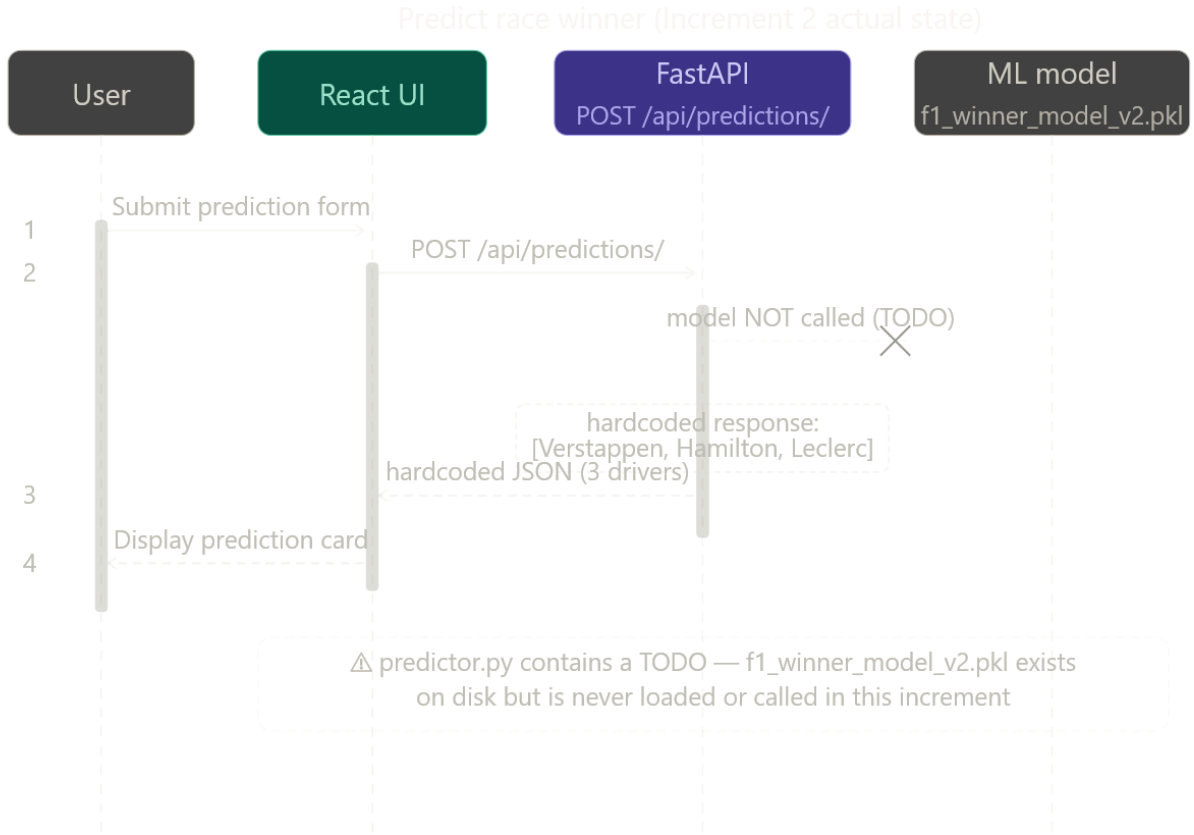
*A **Sequence Diagram** simply depicts **interaction between objects** (or **functions -** in our case - for non-OOP systems) in a sequential order, i.e. the order in which these interactions take place. Sequence diagrams describe how and in what order the objects in a system function*

Use Case diagram depicted below.

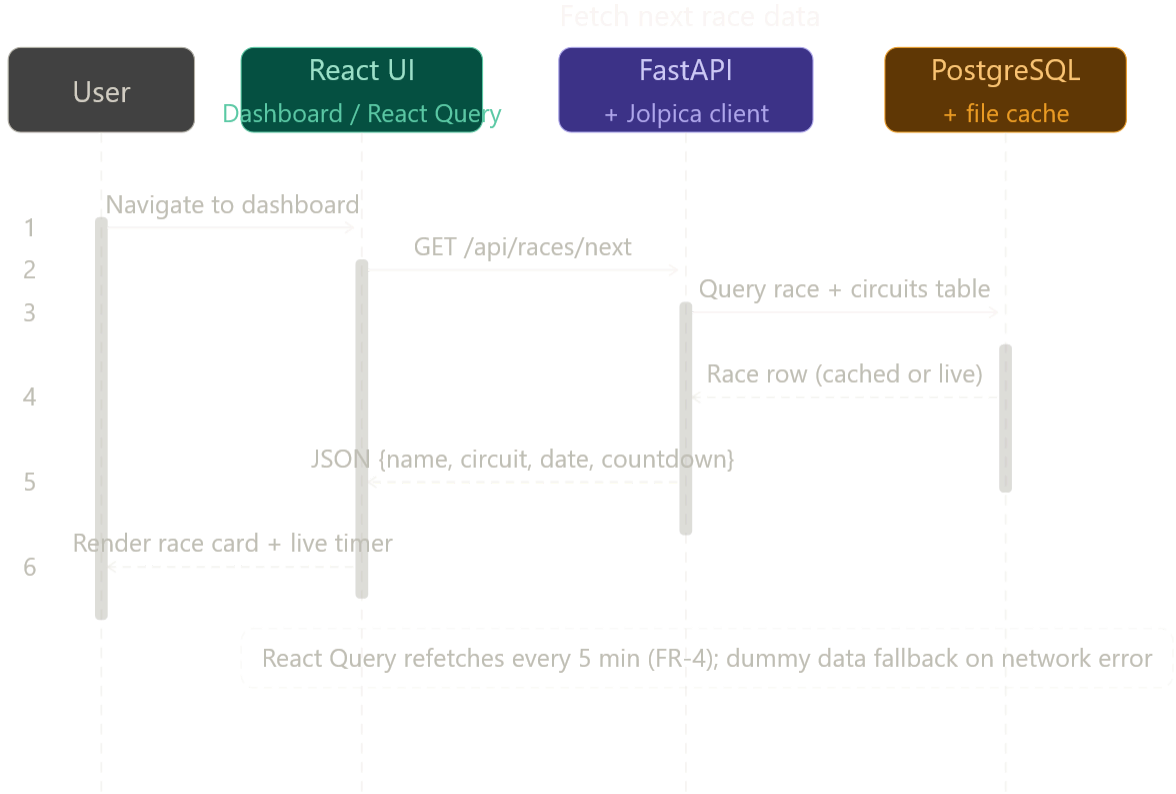


Sequence diagrams pictured below.

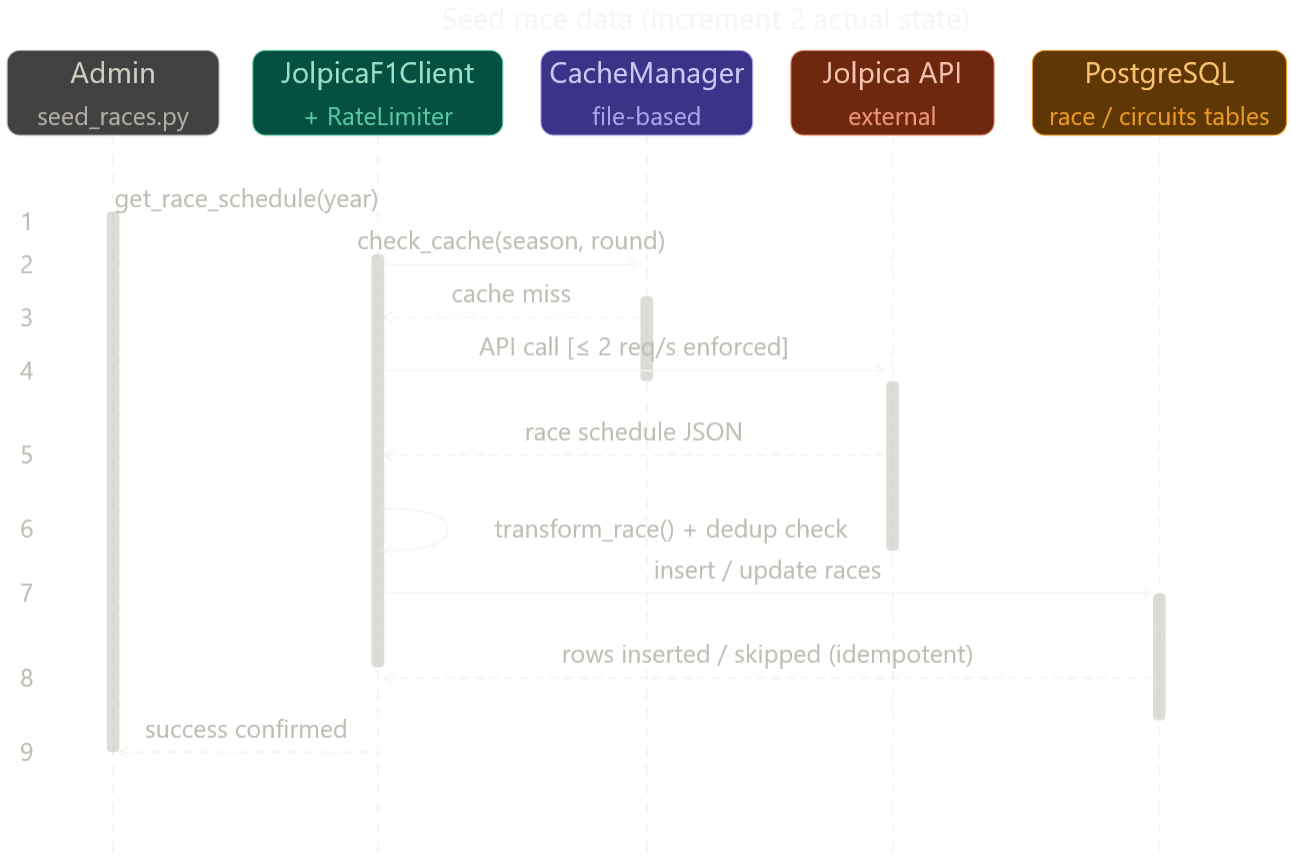
Predict Race Winner sequence — shows hardcoded return; current solution



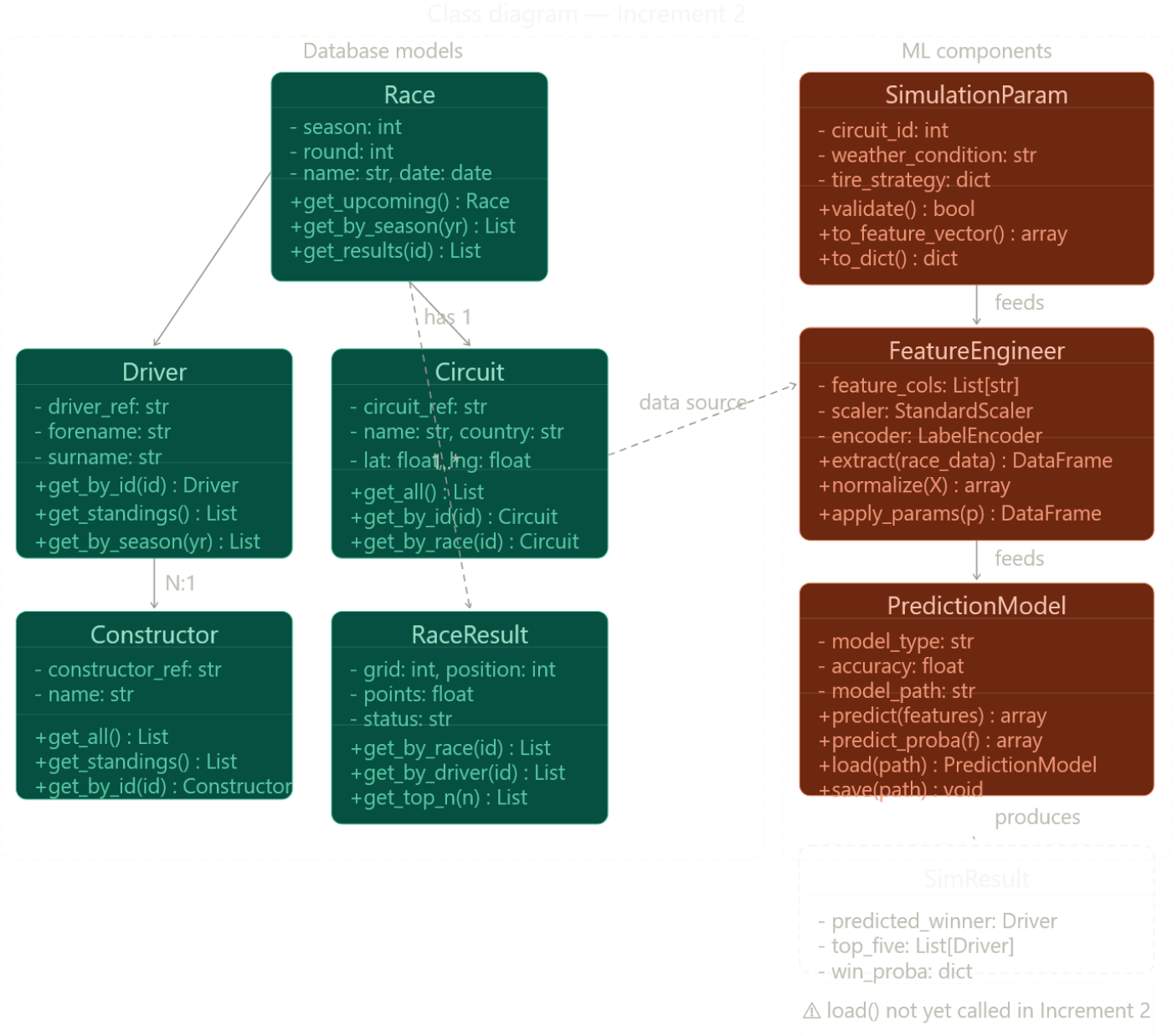
Fetch Next Race Data — corrected to use `/api/` prefix endpoints:



Seed Race Data



And the **class diagram** — existing DB models and ML components only, no ingestion pipeline (that's Increment 3):



6. Operating Environment (5 points)

Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.

- Python 3.12+ — backend API, FastAPI application, ML model execution, data ingestion pipeline, and database seeding scripts
- Node.js 18+ — frontend build toolchain (Vite) and development server (port 5173)
- Modern Web Browser (Chrome, Firefox, Safari, Edge) — client-side execution of the React application
- PostgreSQL 16 — primary relational database server
- FastAPI — REST API framework served on localhost:8000
- Uvicorn — ASGI server for running the FastAPI application
- SQLAlchemy 2.0+ — ORM and database session management
- Alembic — database migration management
- psycopg2 — PostgreSQL adapter for Python
- Pydantic v2 — response model definition and data validation
- pydantic-settings — environment variable loading and backend configuration management
- scikit-learn 1.3+ — ML model training and prediction
- XGBoost — gradient boosting library used in model training
- pandas 2.0+ — data manipulation and feature engineering
- joblib — ML model serialization and deserialization
- Jupyter Notebook — ML workflow development and model documentation
- React 18 — frontend UI library
- TypeScript — static type checking across all frontend code
- Vite — frontend build tool and development server
- Tailwind CSS v4 — utility-first CSS framework for styling and responsive layout
- React Router — client-side routing between application pages
- React Query — server state management and background data fetching
- Axios — HTTP client for frontend-to-backend communication
- Pytest 9.0+ — backend integration and unit testing framework
- Python-dotenv 1.2+ — environment variable management for local development

7. Assumptions and Dependencies (5 points)

List any assumed factors (as opposed to known facts) that could affect the requirements stated in this document. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project.

Assumptions:

- Historical F1 race data from 2010 through 2025 is available, accurate, and sufficient for training a competitive prediction model.
- 2026 season race results will be accessible via the Jolpica F1 and OpenF1 APIs within a reasonable time after each race completes.
- Driver and team performance patterns contain enough signal year-over-year to produce predictions meaningfully better than random baseline.
- The Jolpica F1 API remains publicly available and free to use without authentication requirements throughout the project.
- OpenWeatherMap and Visual Crossing APIs remain available under their current free-tier terms for the duration of the project.

- The application will be accessed primarily via modern desktop browsers; mobile support is desirable but not critical for graded evaluation.

Dependencies:

- **Jolpica F1 API** — primary source for historical and current race schedules, results, driver standings, and constructor standings.
- **OpenF1 API** — supplementary real-time race data source for the automated ingestion pipeline.
- **OpenWeatherMap API** — real-time weather forecasts for race locations used in the Simulator.
- **Visual Crossing API** — historical weather data used as a training feature for the ML model.
- **scikit-learn / XGBoost** — machine learning libraries for model training, feature engineering, and prediction.
- **joblib** — model serialization and deserialization for the trained prediction artifact.
- **PostgreSQL** — primary relational database for all race, driver, circuit, and prediction data.
- **SQLAlchemy 2.0+** — ORM layer for all database interactions from the FastAPI backend.
- **FastAPI / Uvicorn** — backend REST API framework and ASGI server.
- **pydantic-settings** — environment variable management and configuration validation in the backend.
- **React 18 / TypeScript / Vite** — frontend UI library, type system, and build toolchain.
- **React Router** — client-side navigation between Dashboard, Simulator, Data Center, and Newsroom pages.
- **React Query** — server state management and background data fetching in the frontend.
- **Axios** — HTTP client for frontend-to-backend API communication, with interceptors and retry logic.
- **Tailwind CSS v4** — utility-first CSS framework for styling and responsive layout.